通用人工智能创新实践课 - 经典 Agent 设计范式

Agent 架构分层

三层结构把基础能力与可选决策机制分开，也解释了为什么经典方法经常被组合使用

组织协调层 多个决策主体如何分工与通信

角色协作 · Orchestrator–Workers · SOP

单体控制层 一个 Agent 如何分配推理时计算

ReAct · Planning · Feedback · Search

基础能力层 模型如何获得动作、状态与外部证据

Tools / Environment · State / Memory · Loop / Stop

介绍顺序



另一条独立维度

Prompt-only

监督微调

工具使用训练

强化学习 / 在线学习

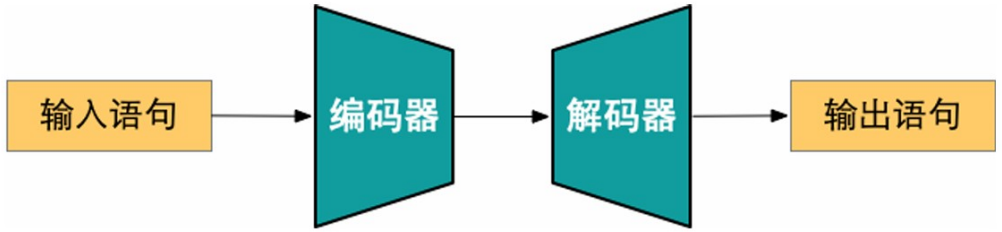
Agent 框架的早期探索

早期对话系统、工具使用与环境

- MRKL： 语言模型作为路由器，连接知识库、计算器和符号推理模块。
- SayCan： 语言模型给出动作的语义适配性，价值函数判断动作在当前环境中是否可执行。
- Toolformer： 通过训练学习何时调用什么工具以及如何使用返回结果。

LLM 之前的 Agent

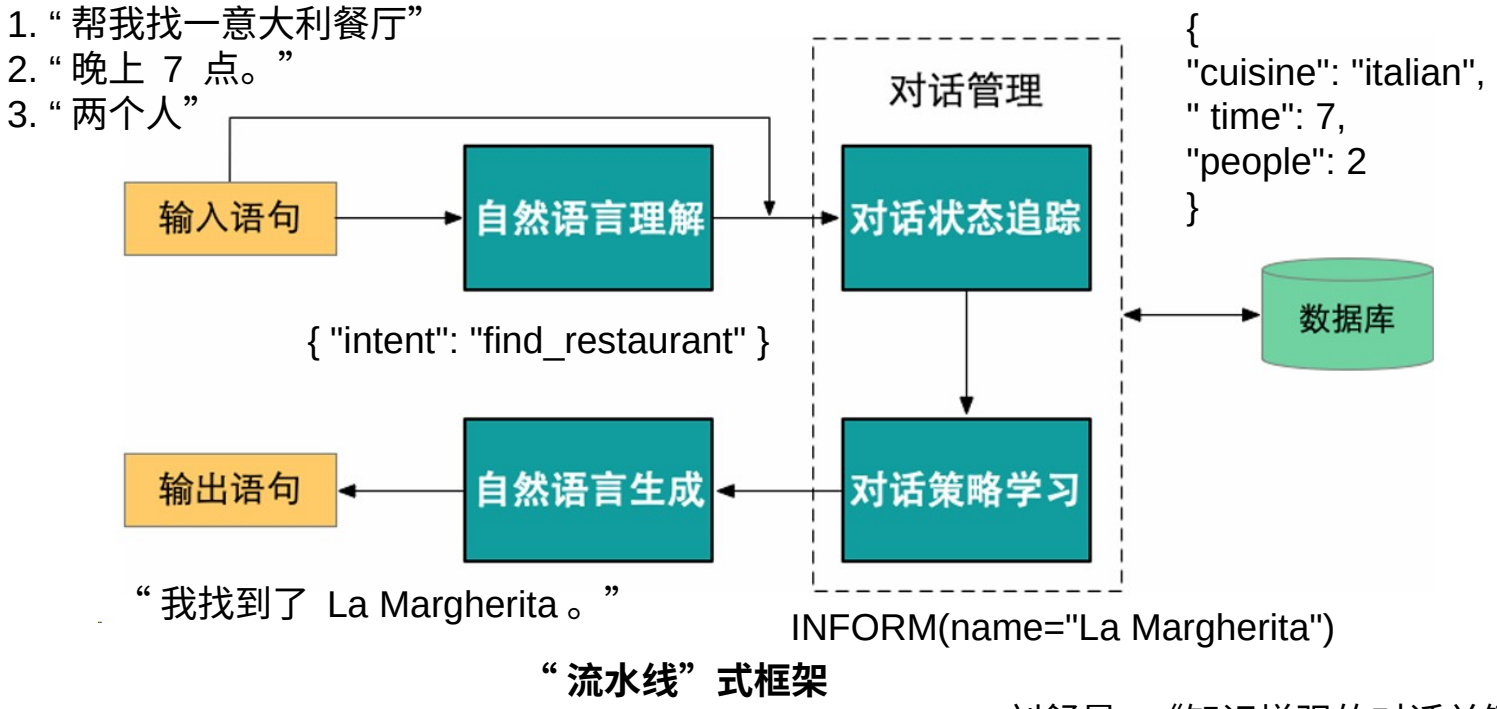
对话系统



Encoder-Decoder Structure

Encoder-Decoder Structure
（Sequence-to-Sequence Structure）

- 编码器将自然语言形式的用户输入编码为包含有对话含义的隐状态
- 解码器根据隐状态和当前已生成的输出序列预测回复内容中的下一个词，并最终生成完整的输出语句



“ 流水线” 式框架

Pipeline Structure

- NLU： NaturalLanguage Understanding
- DST： Dialogue State Tracking
- DPL： Dialogue Policy Learning
- NLG： Natural Language Generation

问题：新领域泛化性弱

刘舒曼 . 《知识增强的对话关键技术研究》 [D]. 中国科学院大学（中国科学院计算技术研究所） , 2023.

MRKL： neuro-symbolic system

解决的问题

- 知识不新鲜
- 无法访问私有知识
- 某些计算不适用神经网络近似
- model explosion
  - ➢ 为每个领域重新 fine-tune 成本高
  - ➢ 灾难性遗忘

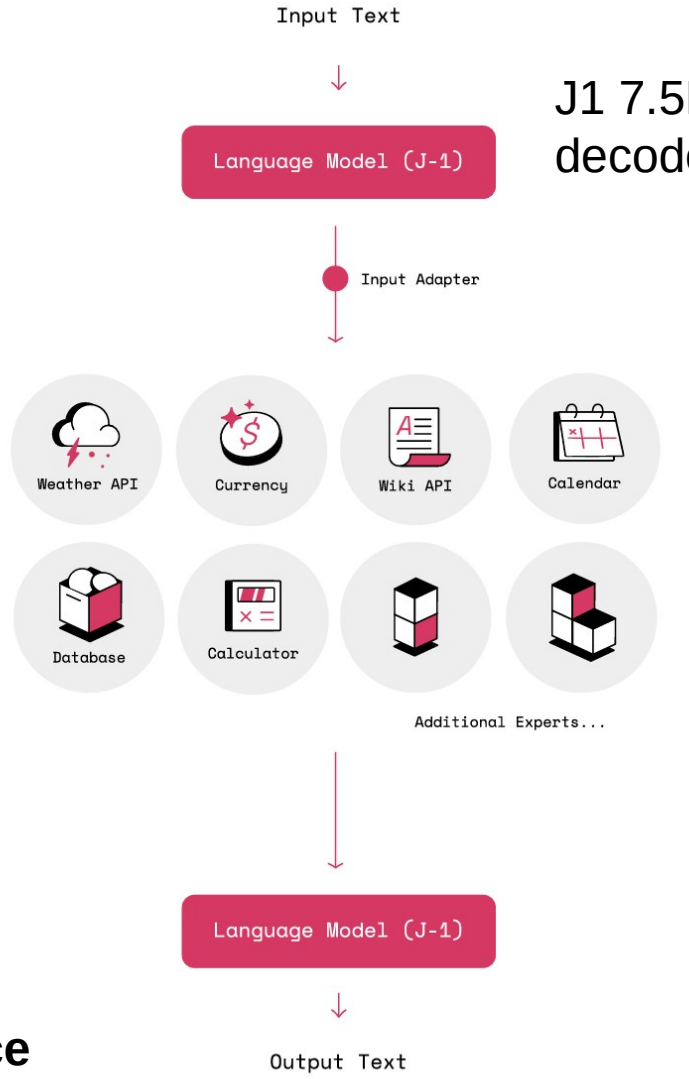
解决思路

MRKL 中语言模型作为 router。它接收用户的自然语言请求，然后决定把请求发送给哪个 expert

Expert 可以是：

- 神经模块：通用 LLM、小型领域模型；
- 符号模块： calculator、 database、 API、 currency converter 等。

Agent 的 intelligence 并不等于 foundation model 的 intelligence



J1 7.5B
decoder-only

Karpas E, Abend O, Belinkov Y, et al. Mrkl systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning[J]. arXiv preprint arXiv:2205.00445, 2022.

SayCan：知识 -> 可执行的决策

LLM 知道很多，但它并不知道自己所处的具体世界，也不知道自己的身体 / 工具到底能做什么。 作者因此把机器人决策拆成两个概率：

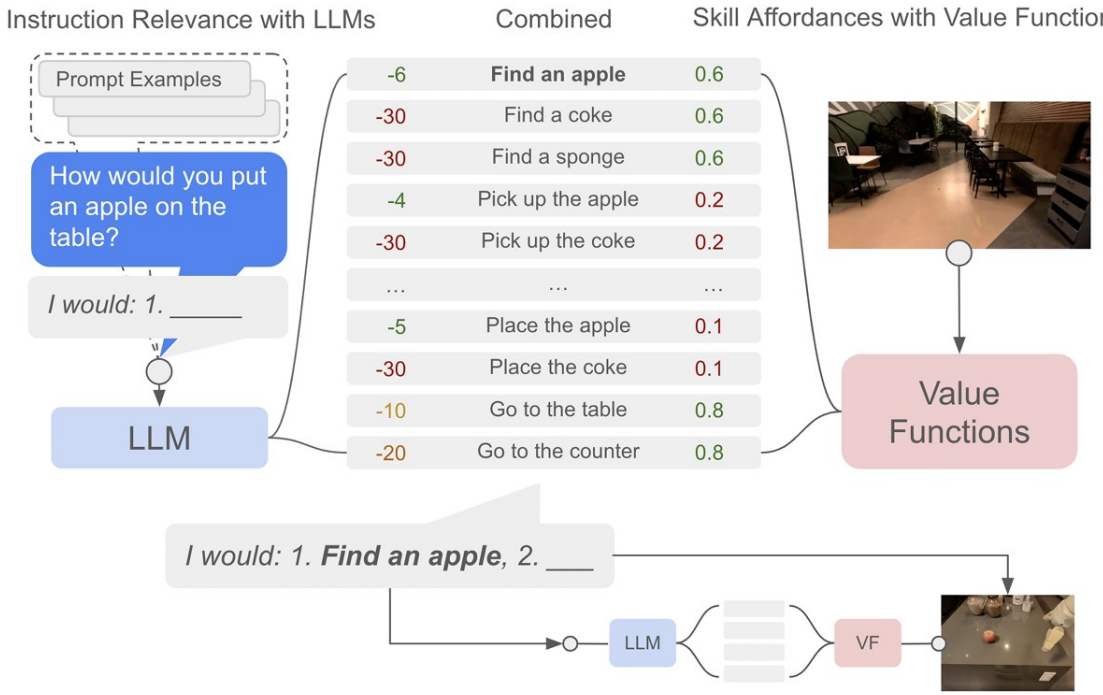
$$\text{Score}(a) = \underbrace{p(a \mid \text{goal}, \text{history})}_{\text{Say：应该做什么}} \times \underbrace{p(\text{success} \mid s, a)}_{\text{Can：实际上能不能做}}$$

然后：

$$a^* = \arg\max_a \text{Score}(a)$$

这里的 Say 来自 LLM， Can 来自机器人技能对应的 value function / affordance function 。 LLM 负责高层语义和任务分解，机器人技能与 value function 则把它约束到现实世界。

仅仅把 LLM 变聪明，并不能自动得到一个可靠 Agent 。



语言模型里的世界知识，怎样变成一个真实 Agent 可以执行的决策?

论文把这个问题拆成两个 grounding：

- Task grounding：
  这个 action 对目标有没有帮助？由 LLM 完成。
- World grounding：
  在现在这个环境、这个 embodiment 下，这个 action 能不能成功？由 value function 完成。

Ahn M, Brohan A, Brown N, et al. Do as i can, not as i say: Grounding language in robotic affordances[J]. arXiv preprint arXiv:2204.01691, 2022.

Toolformer： LLM 自己学习「何时、如何、调用哪个工具」

核心思想：让模型自主学习 何时调用工具、调用哪个工具、以及如何构造参数。

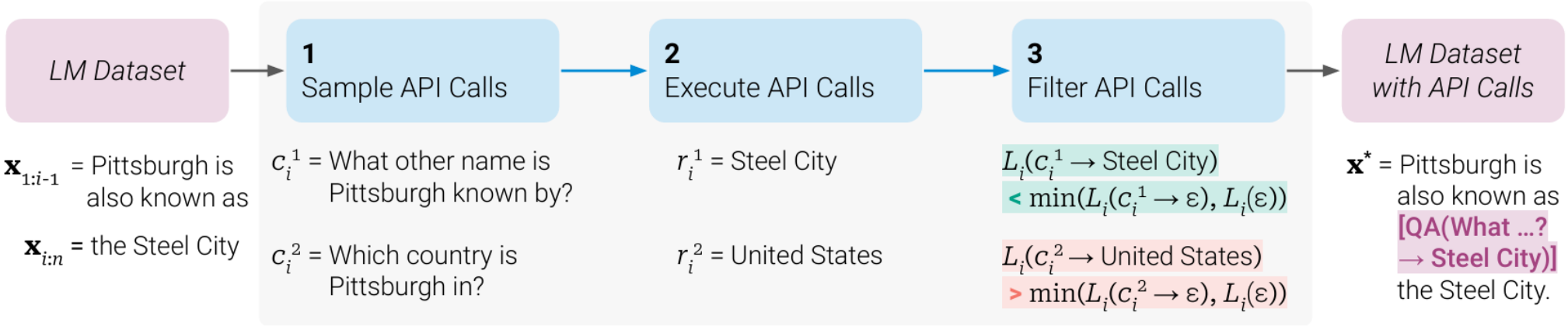
训练方法：
Sample API Calls → Execute → Filter
模型先生成候选工具调用，真实执行 API，再只保留那些能显著降低后续文本预测 loss 的调用：

即：如果工具结果让模型更容易预测后续文本，这次调用就是有价值的。

对今天 Agent 的意义

- **Tool use 成为模型能力**： LLM 不只生成文本，也开始自主选择和执行 action 。
- **奠定 Tool Loop 雏形**：
- **启发自动化 Agent 训练**：模型生成 action ，环境执行，再根据结果筛选高质量 trajectory 。

局限：仍缺少 multi-step planning 、 tool chaining 和 long-horizon credit assignment 。
Toolformer 不是完整 Agent，但它奠定了“ LLM 作为 tool-using policy” 的重要基础。



Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language models can teach themselves to use tools[J]. Advances in neural information processing systems, 2023, 36: 68539-68551.

单 Agent 的四种决策机制

- ReAct
- Plan-and-Execute
- Self-Refine
- 多轨迹搜索

ReAct

Reason-only

只在模型内部展开推理

Thought
分解问题

↓

Thought
继续依赖内部知识

↓

Answer
错误可能沿推理链传播

局限：不能主动获得新证据

Act-only

直接从观察映射到动作

Action
执行环境动作

↓

Observation
获得新的环境状态

↓

Action
缺少目标分解与进度跟踪

局限：长轨迹中容易重复或偏离目标

ReAct

在同一轨迹中交错推理与行动

Thought
判断需要什么信息

↓

Action
调用工具或操作环境

↓

Observation
外部反馈进入上下文

↓

Next Thought
据此调整计划或答案

Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models[J]. arXiv preprint arXiv:2210.03629, 2022.

ReAct

Few-shot trajectory format （示意）

Prompt 中先给出若干完整示例；模型继续生成同一格式

Thought 1: 需要先确定第一个实体。

Action 1: Search[entity A]

Observation 1: ..Wikipedia API 返回的文本 ...

Thought 2: 根据观察，下一步查找实体 B 。

Action 2: Lookup[entity B]

Observation 2: .. 外部环境返回的新证据 ...

Thought 3: 已有足够证据，可以作答。

Action 3: Finish[answer]



实现边界

- Observation 追加进上下文，影响下一次生成
- 知识任务： Thought–Action–Observation 较密集
- 长序列决策：只在关键位置使用稀疏 Thought

Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models[J]. arXiv preprint arXiv:2210.03629, 2022.

ReAct

```python
def react_agent(question):
    context = [f"Question: {question}"]

    while True:
        response = llm(context)

        if response.type == "final":
            return response.answer

        # Reason / Thought
        thought = response.thought

        # Act
        action = response.action
        action_input = response.action_input

        # Observation
        observation = tools[action](action_input)

        context.append(f"Thought: {thought}")
        context.append(f"Action: {action}({action_input})")
        context.append(f"Observation: {observation}")
```

```
Question: 北京今天多少度?

Thought: 我需要查询天气。
Action: get_weather("北京")
Observation: 28°C，多云

Thought: 已经获得所需信息。
Final Answer: 北京今天约 28°C，多云。
```

贡献：

奠定了现代 Agent 的基本交互范式。

ReAct 将 **Reasoning 与 Acting 交错执行**，形成“思考 → 行动 → 观察 → 再思考”的闭环，使 LLM 从一次性生成答案，转向能够调用工具、获取外部反馈并动态调整策略的智能体。

今天主流 Agent 中的 **tool use 、 environment interaction 、 trajectory-based execution** ，都可以看到 ReAct 所确立范式的延续。

局限性：

它更像 Agent 的“最小循环”，而不是完整的 Agent 架构。

ReAct 依赖逐步、串行的局部推理与行动，容易出现错误累积、重复循环和无效工具调用，在长程任务中还面临上下文、成本和稳定性问题。

今天的 Agent 已进一步引入 **显式规划、记忆与状态管理、结果验证 / 反思、并行执行及多 Agent 协作** 等机制—— ReAct 仍是重要的思想起点，但单纯的 ReAct loop 已不足以支撑复杂、长期、高可靠的 Agent 系统。

Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models[J]. arXiv preprint arXiv:2210.03629, 2022.

Plan-and-Execute

维度	Plan-and-Solve	ReWOO	HuggingGPT
主要任务	静态多步推理	工具增强问答	跨模型、多模态任务
计划表示	自然语言子问题	带 #E 变量的依赖计划	任务列表与模型描述
观察依赖	求解时无外部环境	Planner 不读取执行观察	执行后汇总各子任务结果
执行主体	同一 LLM	Worker 调用工具， Solver 汇总	ChatGPT 选择并调度模型
主要收益	减少遗漏步骤	减少重复 LLM 调用与 token	统一管理不同领域模型
主要边界	并非完整交互 Agent	早期计划错误可能传播	模型描述与依赖解析必须可靠

Plan-and-Solve

从

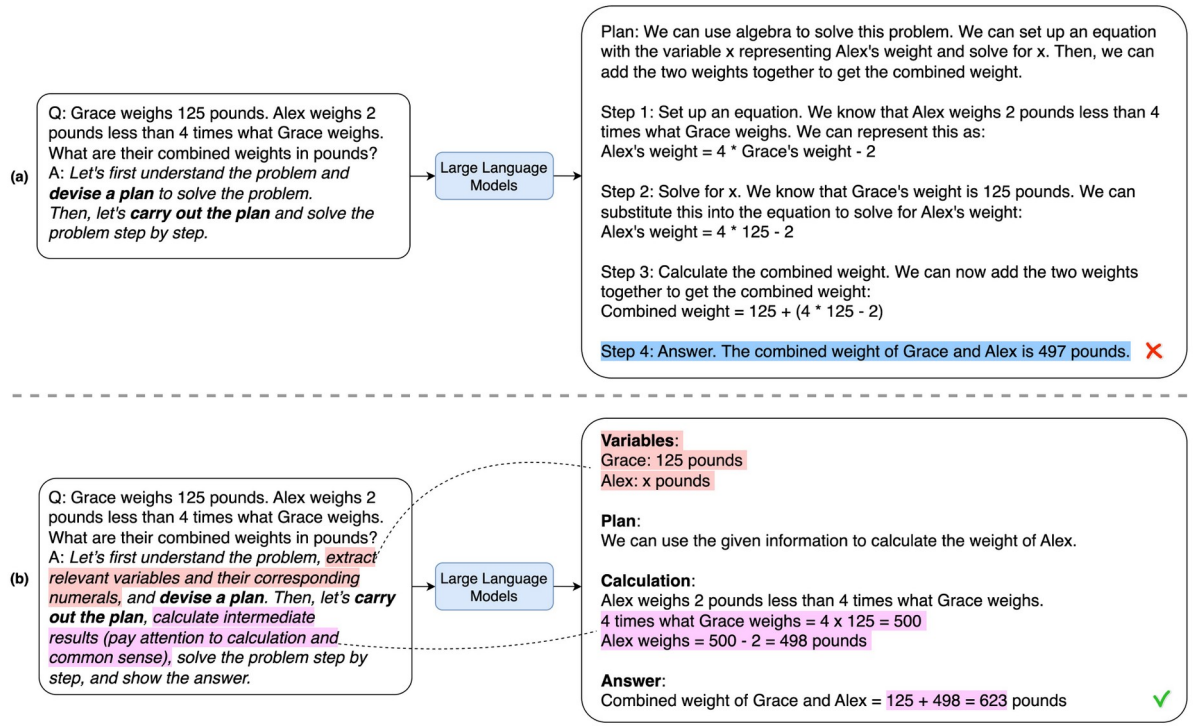
think step by step(CoT)

到

先理解问题并制定一个 plan ，再按照这个 plan 一步一步执行。 (PS)

再到

首先，我们要弄清楚问题的实质，**提取出相关的变量**及其对应的数值，并制定一个**方案**。然后，我们就要执行这个方案，计算**中间结果**（注意计算过程和常识），逐步解决问题，并给出答案。（ PS+ ）



贡献：

1. 从 CoT 到任务分解
2. Planner / Executor Separation
3. Plan 变成一个可以操作的中间对象（可控性、可审计）

局限性：

仍是静态的 **Plan once, Execute once** ，缺少环境反馈、动态 Replan 、工具调用与验证机制，难以应对真实的长程任务。

Wang L, Xu W, Lan Y, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models[C]//Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers). 2023: 2609-2634.

Plan-and-Solve

```python
def plan_and_solve(question):
    context = [f"Question: {question}"]

    # Plan
    plan = llm_plan(context)
    context.append(f"Plan: {plan}")

    results = []

    # Solve
    for step in plan.steps:
        result = llm_solve(
            question=question,
            step=step,
            previous_results=results
        )

        results.append(result)
        context.append(f"Step: {step}")
        context.append(f"Result: {result}")

    # Final
    answer = llm_final(
        question=question,
        plan=plan,
        results=results
    )

    return answer
```

```
Question:
一家商店有 120 个苹果，卖掉了 35%，之后又进货 20 个，现在有多少个苹果？

Plan:
1. 计算卖掉了多少个苹果。
2. 计算卖掉后剩余多少个苹果。
3. 加上新进货的 20 个苹果。
4. 给出最终答案。

Step 1:
120 × 35% = 42
Result: 卖掉了 42 个苹果。

Step 2:
120 - 42 = 78
Result: 剩余 78 个苹果。

Step 3:
78 + 20 = 98
Result: 现在有 98 个苹果。

Final Answer:
现在一共有 98 个苹果。
```

ReWOO

ReAct 的问题：

1. **LLM 和工具调用是串行耦合的。**
2. 所以如果任务越来越长， context 会不断增长。

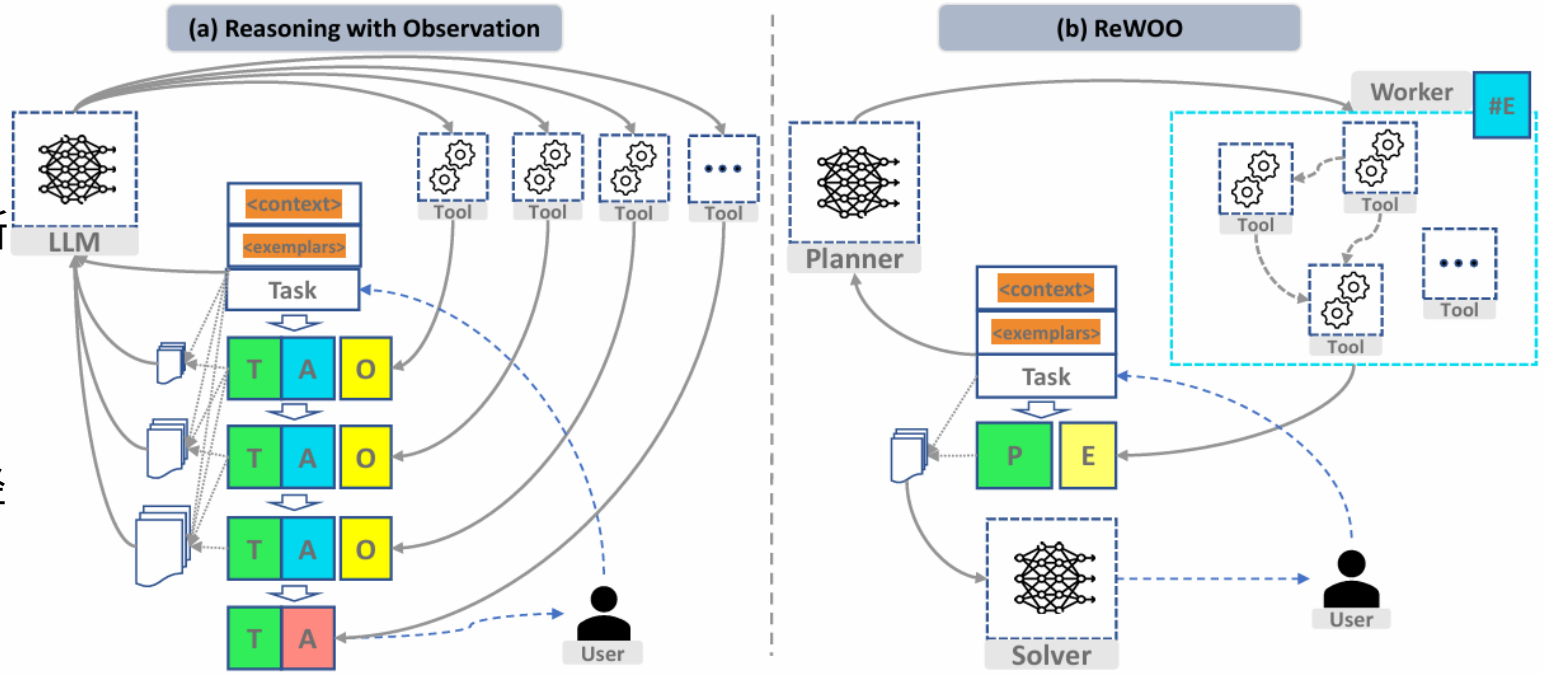
ReWOO 假设：

LLM 在真正看到 observation 之前，就已经可以预判一个任务大致需要哪些步骤。（**Foreseeable Reasoning**）

于是它把 Agent 拆成了三个模块：

- **Planner** 一次性产生完整 blueprint
- **Worker** 根据 blueprint 调用工具收集 evidence
- 最后 **Solver** 把计划和 evidence 一起综合成答案。

ReWOO 还有一个非常超前的思想：不同模块可以用不同模型



对今天 Agent 的意义

- Planner / Worker / Solver 的模块解耦
- 把 cost / latency 当作 Agent architecture 的一等公民

稍有过时的概念

- Token duplication 问题没有 2023 年那么严重（前缀缓存、 KV-cache optimizations ）

Xu B, Peng Z, Lei B, et al. Rewoo: Decoupling reasoning from observations for efficient augmented language models[J]. arXiv preprint arXiv:2305.18323, 2023.

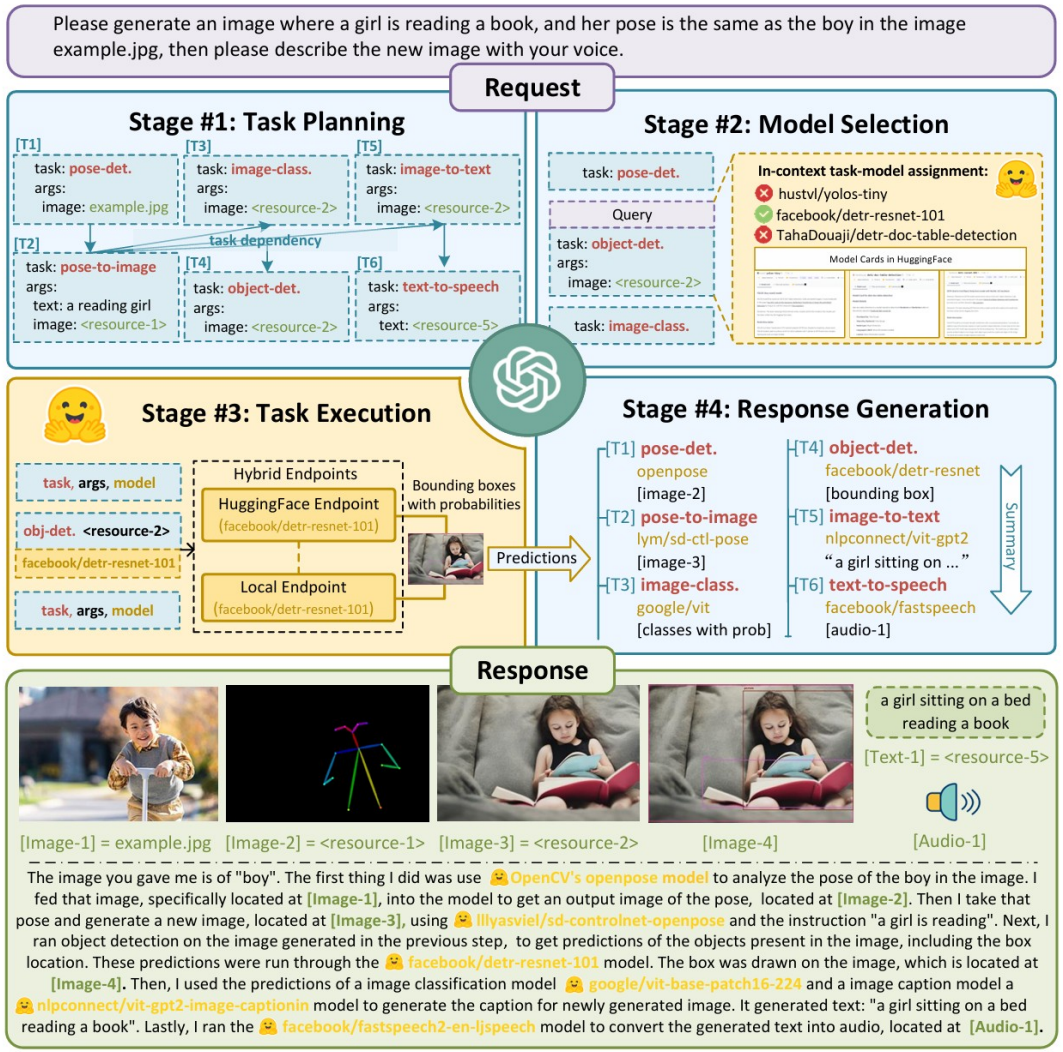
HuggingGPT

LLM 很擅长：
- 理解自然语言；
- 推理；
- 分解任务；
- 与用户交互。

但 2023 年的 ChatGPT 不擅长：
- 图像检测；
- 图像生成；
- pose detection；
- speech recognition / TTS；
- 各种高度专业化的任务。

而 Huggingface 上有大量专用模型，于是 HuggingGPT 提出的核心思想是：

不要让一个模型完成所有事情。让 LLM 做“大脑 / Controller”，让大量 expert models 做“手脚 / Executors”。



对今天 Agent 的意义
- LLM 作为控制器，外部模型 / 工具作为执行器
- 已经实现简单 DAG scheduler

Shen Y, Song K, Tan X, et al. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face[J]. Advances in Neural Information Processing Systems, 2023, 36: 38154-38180.

Self-Refine

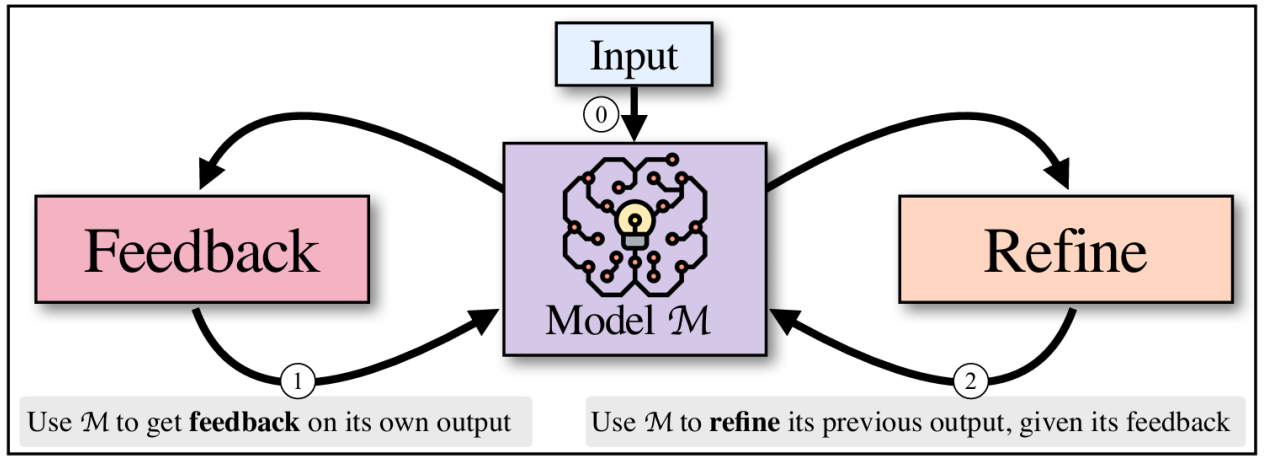
基本结构

1）假设用户给定输入，模型 先正常产生一个答案：

2）不直接把 返回给用户，而是让**同一个** LLM 切换角色，对自己的答案提出反馈：

3）然后再让同一个 LLM 根据这个 feedback 修改答案：

不断循环。



对今天 Agent 的意义

- Evaluator-Optimizer Agent 架构
- Test-Time Scaling

但目前实践中，最好的 Self-Refine，其实已经不是“Self” Refine 了，而是类似于 coding agent 这种通过确定性的执行来获取反馈。

ReAct 告诉我们 LLM 应该“边想边行动”
Self-Refine 告诉我们 LLM 应该“**做完以后检查并修改**”

Madaan A, Tandon N, Gupta P, et al. Self-refine: Iterative refinement with self-feedback[J]. Advances in neural information processing systems, 2023, 36: 46534-46594.

CRITIC

假设 LLM 生成了答案：

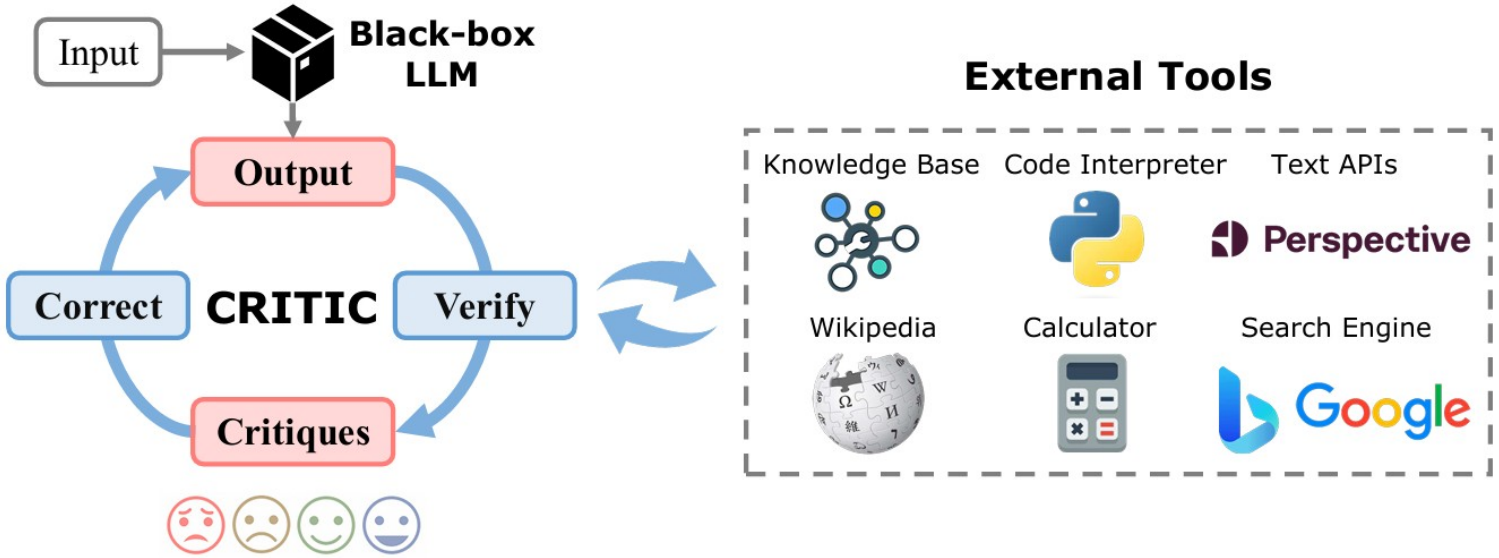
self-refine 做法大概是：

问题在于：

产生错误答案的那个模型，未必有能力判断这个答案是错的。

CRITIC 提出：

基于外部工具反馈产生的 critique



对今天 Agent 的意义

- 纯粹靠 prompted LLM 自纠错并不可靠
- Tool 不仅可以扩展能力，也能获取反馈

CRITIC 是现代 coding agent “execute–inspect–repair” loop 的一个非常早期、非常干净的理论原型。

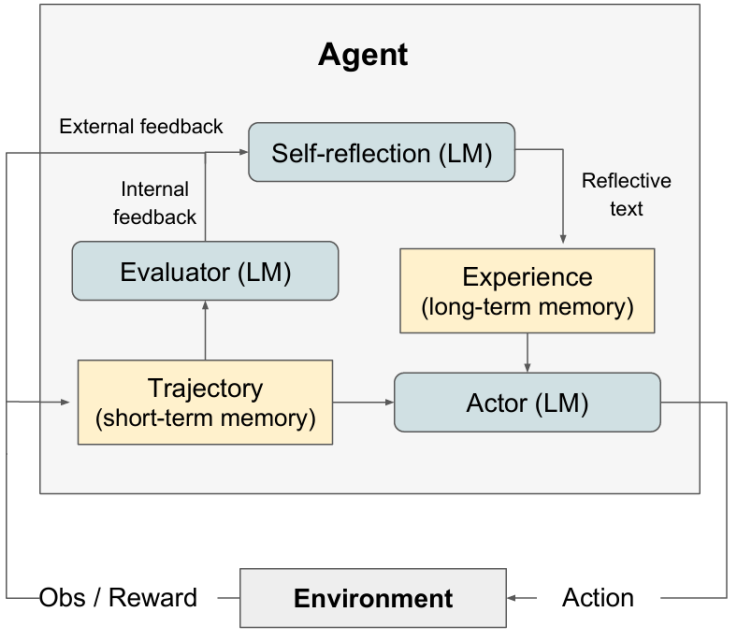
Gou Z, Shao Z, Gong Y, et al. Critic: Large language models can self-correct with tool-interactive critiquing[C]//International Conference on Learning Representations. 2024, 2024: 57734-57811.

Reflexion

如果 Agent 失败了，怎么办?
1. 再试一次 -> 相同的错误
2. RL -> 成本太高

Reflexion 提出了一条路径:

不更新 weights，更新 context。
论文把这个称为 **Verbal Reinforcement Learning （语言强化学习）**



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection: M_a, M_e, M_{sr}
Initialize policy $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$
Generate initial trajectory using π_θ
Evaluate τ_0 using M_e
Generate initial self-reflection sr_0 using M_{sr}
Set $mem \leftarrow [sr_0]$
Set $t = 0$
while M_e not pass or $t <$ max trials **do**
  Generate $\tau_t = [a_0, o_0, \ldots a_i, o_i]$ using π_θ
  Evaluate τ_t using M_e
  Generate self-reflection sr_t using M_{sr}
  Append sr_t to mem
  Increment t
end while
return

- **Actor** ： 产生 reasoning / action
- **Evaluator** ： 判断这次 trajectory 是否成功
- **Self-Reflection** ： 把失败和 reward 转化成自然语言经验
- **Memory mem**： 保存这些经验

policy=model weights+experience / context

对今天 Agent 的意义
- inference-time learning 的 Agent 范式
- Actor–Critic/Verifier Agent Loop
- Memory 从“聊天记录”变成了“经验”

Shinn N, Cassano F, Gopinath A, et al. Reflexion: Language agents with verbal reinforcement learning[J]. Advances in neural information processing systems, 2023, 36: 8634-8652.

Reflexion

```python
def reflexion_agent(task, max_trials=3):
    memory = []

    for trial in range(max_trials):
        # Attempt
        trajectory, answer = run_agent(
            task=task,
            memory=memory
        )

        # Evaluate
        feedback = evaluate(
            task=task,
            answer=answer
        )

        if feedback.success:
            return answer

        # Reflect
        reflection = llm_reflect(
            task=task,
            trajectory=trajectory,
            feedback=feedback
        )

        # Store reflection for next trial
        memory.append(reflection)

    return answer
```

```
Task: reciprocal(x), x=0 时返回 None

Trial 1
Attempt:
return 1 / x

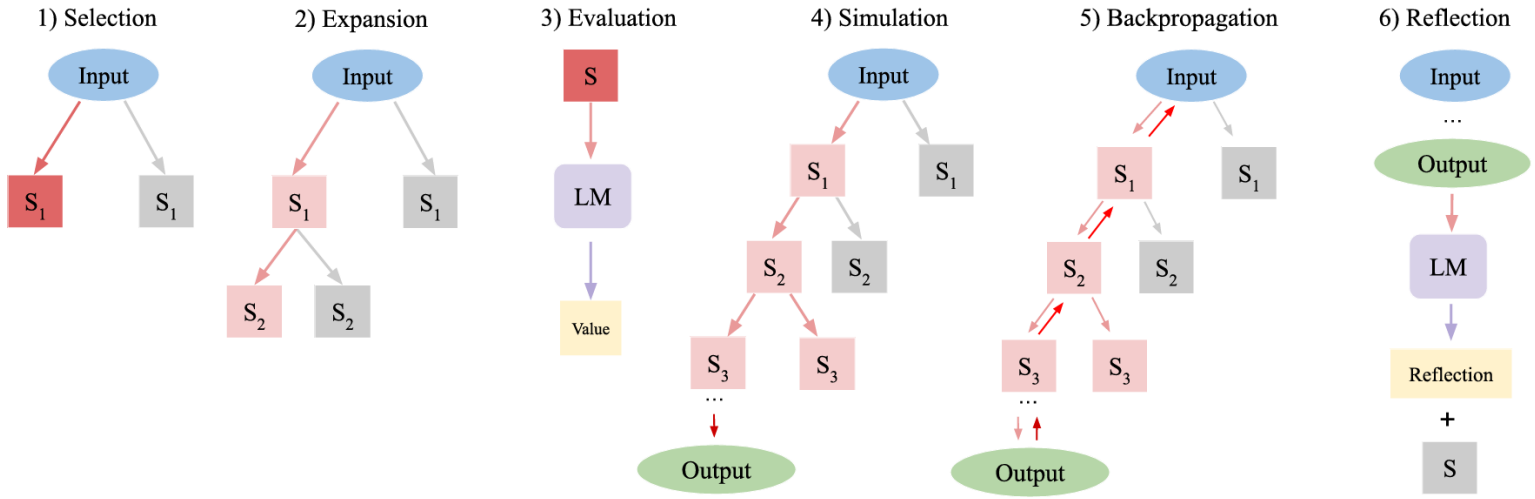
Feedback:
x=0 -> ZeroDivisionError

Reflection:
我忽略了 x=0 的边界情况，
下一轮先判断 x == 0。

Trial 2
Attempt:
if x == 0:
    return None
return 1 / x

Result:
PASS
```

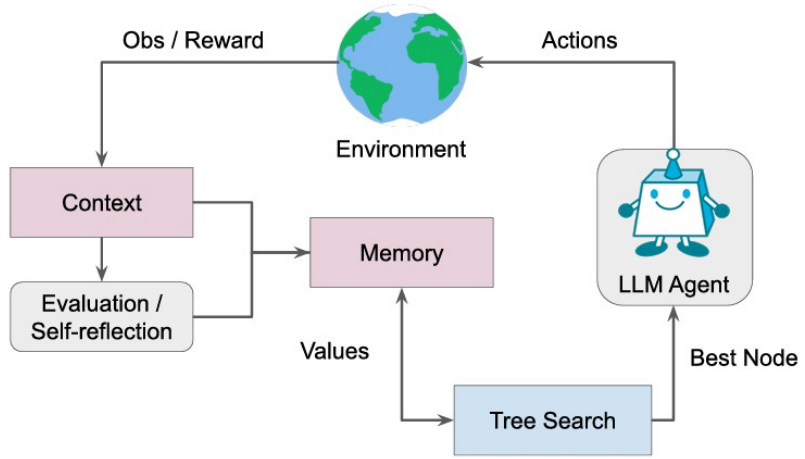
LATS



ReAct 中，如果第二步出错了，后面的 trajectory 很容易一路错下去。

用树搜索统一 Agent 的规划与执行

- **多分支探索**
  每个状态生成多个候选 Action，通过树搜索比较不同执行路径，而非沿单一路径持续推进。
- **环境反馈驱动决策**
  Action 实际作用于 Environment，基于返回的 Observation 评估节点价值，并决定继续探索、回退或切换分支。
- **失败反思指导后续搜索**
  对失败 trajectory 生成 Reflection，将错误原因与改进建议加入上下文，影响之后的行动生成与价值判断。



对今天 Agent 的意义

- 环境反馈是关键监督信号
- Test-time compute 可以用于探索多条 trajectory
- 规划与真实执行应适当解耦

Zhou A, Yan K, Shlapentokh-Rothman M, et al. Language agent tree search unifies reasoning acting and planning in language models[J]. arXiv preprint arXiv:2310.04406, 2023.

Agent 范式

- Prompt
- Context
- Harness
- Loop
- Graph

Agent 工程范式时间线

2020-2022

- GPT-3
- Prompt Programming
- CoT

提示开始成为调用通用模型能力的程序接口

2022—2023

- ReAct
- Reflexion
- BabyAGI
- Voyager

行动循环、反馈、外部记忆、任务队列、技能积累

2023—2024

- AutoGen
- DSPy
- AlphaCodium
- LangGraph

多 Agent、计算图、流程工程、自动提示优化在并行发展

2025 年 6 月

- Tobi Lütke 、 Harrison Chase 、 Karpathy 、 Simon Willison 围绕 Context Engineering 发声

已存在的实践获得一个更容易传播、也更适合表达系统复杂度的名称

2025 年末— 2026 年初

- Anthropic 长任务 harness 文章
- OpenAI Harness Engineering 实践

注意力转向跨会话连续性、执行环境、反馈与工程约束。

2026 年 6—7 月

- Loop Engineering 、 Loopcraft 、 Graph Engineering 集中传播

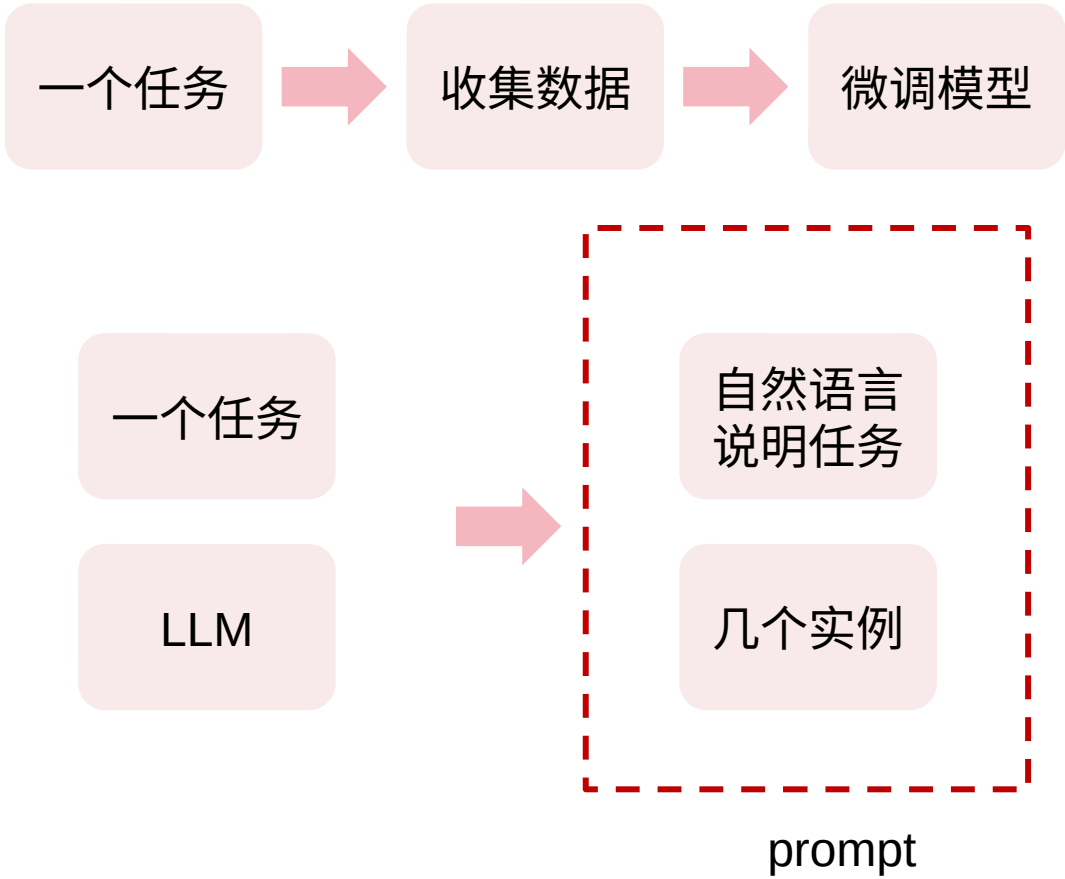
对持续运行、自动触发、多 Agent 编排的讨论被重新包装

Prompt Engineering

背景：
- 模型开始能从上下文中理解任务

相关工作
- Language Models are Few-Shot Learners （GPT-3）
  - in-context learning / few-shot
  - 部分过去由训练过程承担的任务指定，被移动到了推理时的输入里
- Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm
  - 当作一种程序设计讨论，涉及任务分解、叙事框架和生成提示的元提示，而不是简单搜集“更有效的问法”
- Chain-of-Thought

解决的问题：**面对一个能力很强、但行为不完全确定的通用模型，如何表达任务、约束、示例和输出要求，让能力稳定地被调用出来?**



Context Engineering

背景：

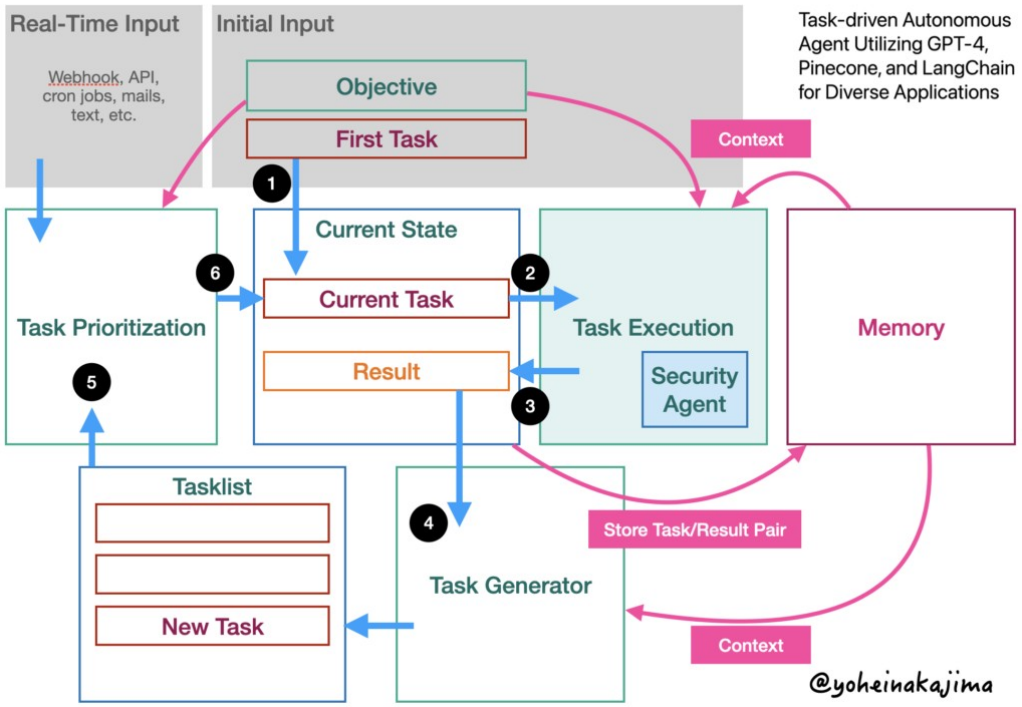
- 从单次 LLM 调用走向循环运行的 Agent；模型持续与外部环境交互，获取新信息、调用工具并执行动作，上下文也随运行过程不断变化

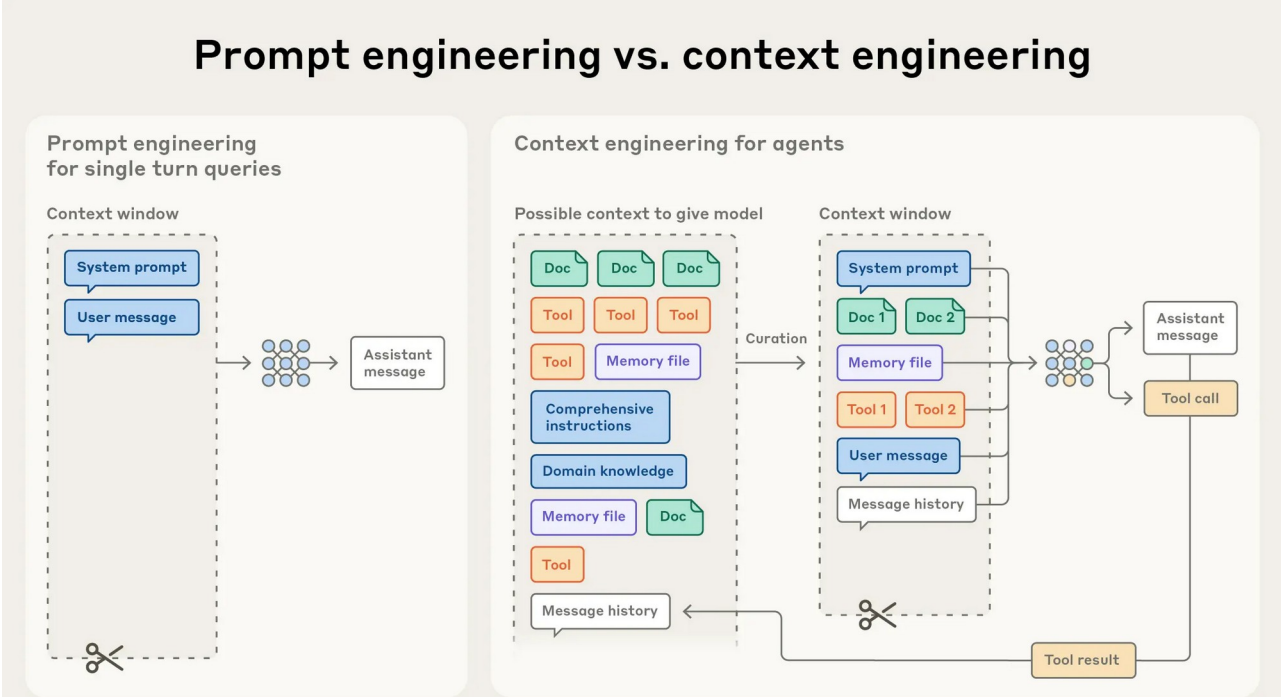
代表工作： ReAct 、 BabyAGI 、 Voyager

Context： LLM 在一次推理中能够访问的全部 token ，包括指令、历史、检索结果、工具输出与工作状态

Context Engineering： 在 Agent 运行过程中，动态构造、管理和维护“最合适的 token 集合”的一系列策略
核心问题是：**每一步应该看到什么、不应该看到什么，以及这些信息应该如何组织?**

系统不再依赖人工一次性写好一个 Prompt ，而是帮助 Agent 持续获取、筛选、压缩和组织上下文，让每一步推理都拥有完成当前任务所需的“ just the right information”





Harness Engineering

Agent = Model + Harness

Harness Engineering 是围绕基础模型设计和持续改进运行时系统，**使模型能够作为可靠、可控、可验证的 Agent 完成真实任务的工程实践。**

Context Engineering：工程化 LLM 的“认知条件”
Harness Engineering：工程化 Agent 的“行为闭环”

在 coding agent 中， harness 泛指一个 coding agent 所工作的完整环境，包括：仓库结构、开发环境、测试、文档、可观测性和反馈机制。确保 agent 可以安全、持续、可恢复地把事情做完。



Loop Engineering

在建立了 harness 后，开发者的工作循环变成了：
1. 创建任务
2. 触发 Agent
3. 检查结果 / code review

“人”逐渐成了这个 loop 当中的卡点和瓶颈

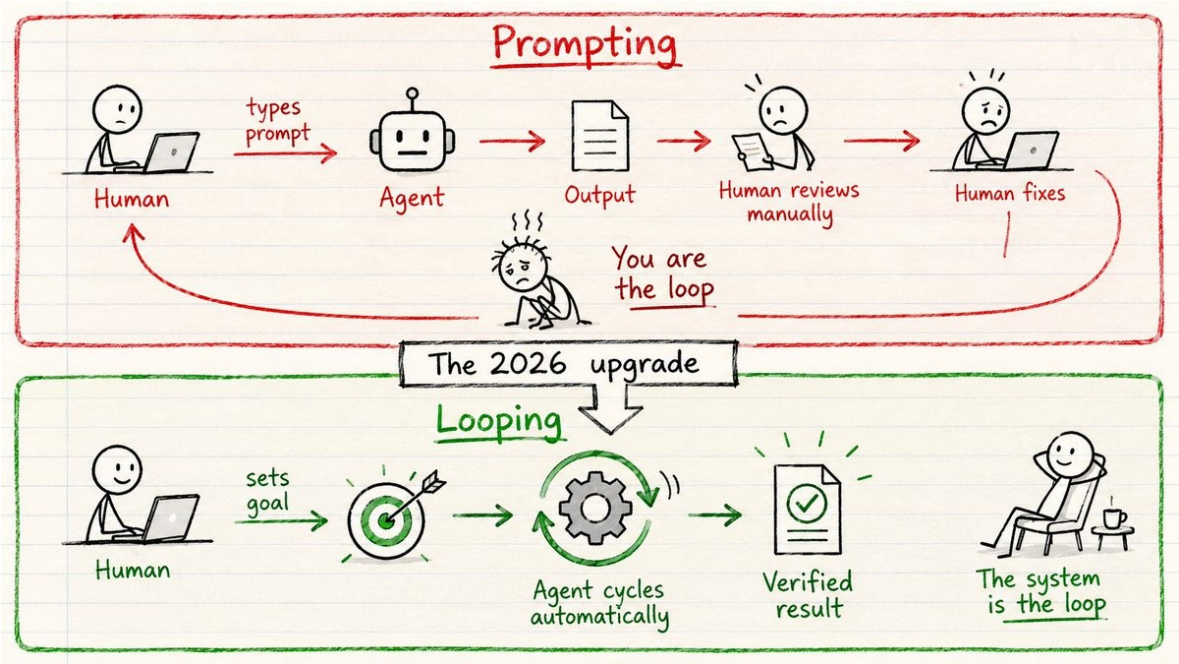
为此， Loop Engineering 的使命是：

设计系统来代替你来引导智能体，而不是由你来执行指令

有效性?
收益 vs 代价？认知投降?

没有明确证据证明有效性
https://arxiv.org/abs/2608.21884

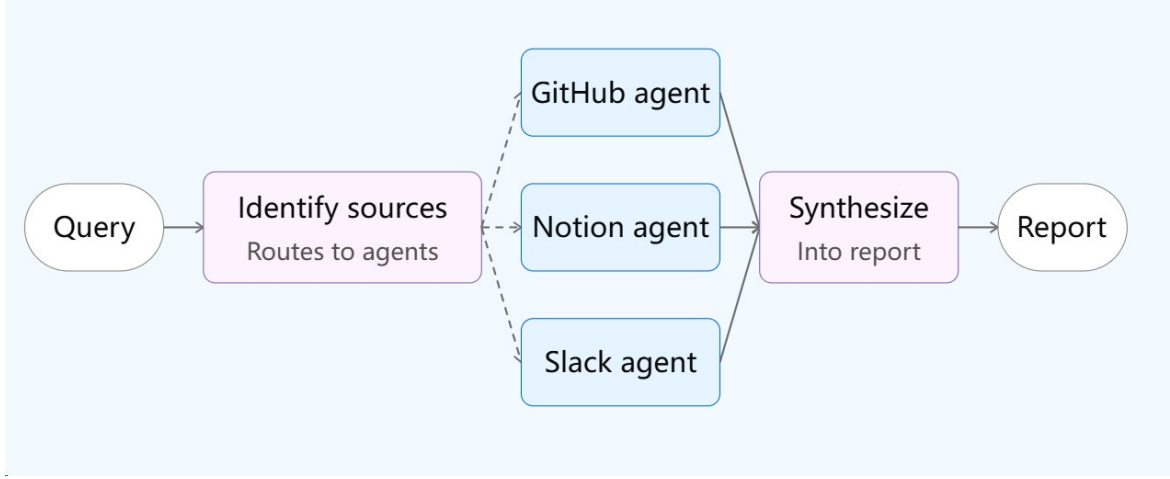
回路	改变的对象	核心问题
工具执行循环	当前任务的下一步动作	现在应该做什么?
验证循环	当前产物	结果合格了吗？不合格如何修正?
事件驱动循环	下一次任务的启动	什么事件、时间或状态需要触发工作?
系统改进循环（自进化）	Prompt 、工具或 Harness 本身	多次失败揭示了什么系统性缺陷?



Graph Engineering

- AutoGen： 多智能体 agent 循环图
- langgraph vs langchain
- 多个 loop 在一个 graph 中分工

层级	你主要设计什么	例子
Prompt Engineering	一条指令	prompt/template
Context Engineering	一次模型调用看到什么	AGENTS.md、 CLAUDE.md
Harness Engineering	一次 agent run 怎么运行	tools、 skills、 subagents、 hooks、 MCP
Loop Engineering	很多次 agent run 如何长期运转	trigger、 stop condition、 state、 verifier、 budget
Graph Engineering	?	?



agents as graphs



常见 Benchmark：从问答到交互决策

Benchmark	一句话介绍	例子
HotpotQA	多跳问答基准，评估模型跨多个文档整合信息并完成复杂推理的能力。	例如：先找到某位导演执导的电影，再判断其中哪部电影获得过奥斯卡奖。
FEVER	事实验证基准，评估模型基于检索证据判断陈述真实性的能力。	例如：判断“埃菲尔铁塔位于伦敦”这一陈述是否正确，并给出支持证据。
ALFWorld	文本化具身智能基准，评估模型在家庭环境中理解任务、规划并执行多步操作的能力。	例如：完成“把苹果放进冰箱”这一任务，需要先找到苹果、拿起苹果并移动到冰箱。
WebShop	模拟电商网站交互基准，评估模型根据用户需求进行商品搜索、比较与购买决策的能力。	例如：根据“购买一双 50 美元以内的黑色跑鞋”这一需求完成搜索、筛选和选购。
HumanEval	代码生成基准，评估模型根据函数说明生成可正确通过测试的程序代码的能力。	例如：根据“输入整数列表，返回其中所有偶数”这一函数说明生成正确的 Python 代码。

指标	基本含义	主要限制
Exact Match	预测与标准答案规范化后完全一致	对等价表述敏感
Token F1	预测与答案 token 的重叠	不保证事实或推理过程正确
Accuracy	标签或最终答案正确比例	不反映证据与过程
Success Rate	完整 episode 达成目标的比例	依赖环境终止规则
Reward / Score	部分满足任务条件的连续得分	不同环境不可直接比较
pass@k	k 个候选中至少一个通过测试的概率	k 越大，推理成本越高

课后习题

作业目标

在现有 cli-sample 基础上，为 TongAgents StatelessReactAgent 新增 arXiv 论文检索工具，使 Agent 能通过 ReAct 流程完成“检索论文 → 基于摘要分析 → 生成中文综述”。

核心任务

- 新增 search_arxiv 工具，支持英文关键词检索及 1–10 条结果限制
- 调用 arXiv API，解析论文标题、作者、日期、摘要、分类及 arXiv/PDF 链接
- 将工具注册到 ToolManager，并配置给 StatelessReactAgent
- 优化 System Prompt：论文综述请求必须先检索，再基于工具结果生成内容
- 输出中文 Markdown 综述，包括：**检索范围、研究脉络、方法分类、论文比较、局限与未来方向、参考论文**

关键要求

请求需设置 timeout、User-Agent；正确处理网络异常、空结果和 XML 解析错误；只能引用工具实际返回的论文，明确说明“证据仅来自论文元数据和作者摘要”，不得凭模型记忆补充或编造文献。

最终效果

用户输入论文主题后，CLI 自动完成：

用户主题 → Agent 调用 search_arxiv → 获取论文摘要 → ReAct 继续生成 → 输出带 arXiv 引用的中文综述